PROYECTO INTERMODULAR

# Fase 2 · Actividad 4

## Diseño técnico, definición del MVP y planificación

Ejemplo completo de referencia

| | |
|---|---|
| **Centro** | IES Albarregas |
| **Ciclo** | CFGS Desarrollo de Aplicaciones Web (DAW) |
| **Curso** | 2.º |
| **Módulo** | Proyecto Intermodular de Desarrollo de Aplicaciones Web (0616) |
| **Fase** | Fase 2 · Diseño y planificación del proyecto |
| **Proyecto ficticio** | Materialia Joven |
| **Equipo ficticio** | Nerea, Hugo y Sara |
| **Continuidad** | Ejemplos de las Actividades 1, 2 y 3 |

> **Importante**
> Este documento muestra una posible forma de resolver la Actividad 4. Las tecnologías, la arquitectura y el modelo de datos responden a las necesidades de este proyecto ficticio: **no son una estructura técnica que todos los proyectos deban reproducir**. Cada equipo debe tomar decisiones coherentes con su propia solución.

## Punto de partida

La Actividad 3 dejó definido cómo será Materialia Joven para sus usuarios: seis áreas, un mapa de navegación por rol, seis wireframes, tres recorridos principales (reservar, entregar y devolver) y un mockup de la ficha de material. También dejó dos cuestiones abiertas:

- RF02 se amplió con un buscador por texto.
- Al bloquear un material con incidencia, había que decidir qué pasa con sus reservas futuras.

En esta actividad se decide **cómo se va a construir** la aplicación y **qué se desarrollará primero**. La segunda cuestión se resuelve en el apartado 6.2.

# 1. Casos de uso principales

El diagrama recoge qué puede hacer cada actor en el sistema. No incluye acciones de interfaz como «pulsar un botón», ni operaciones internas como «guardar en la base de datos»: un caso de uso es algo que el usuario quiere conseguir.

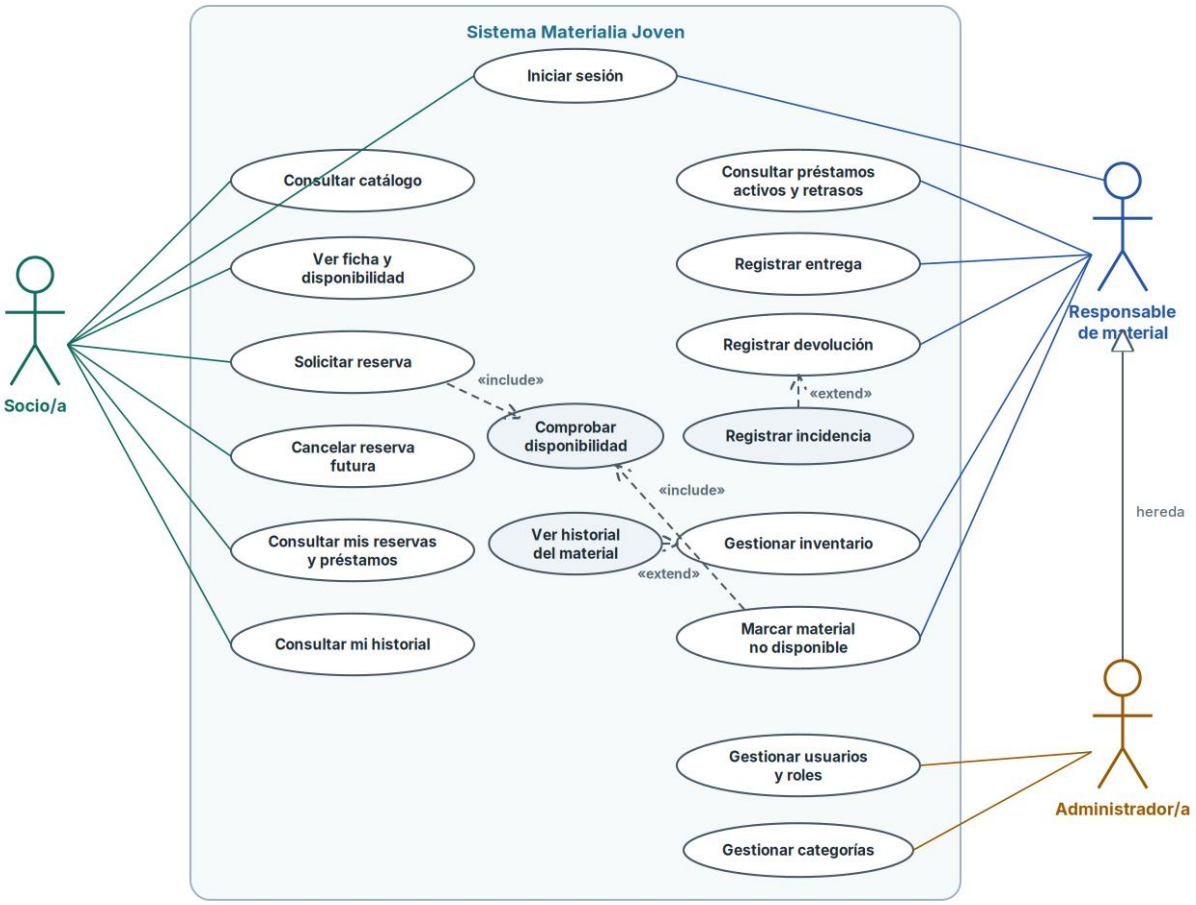



*Diagrama de casos de uso principales*

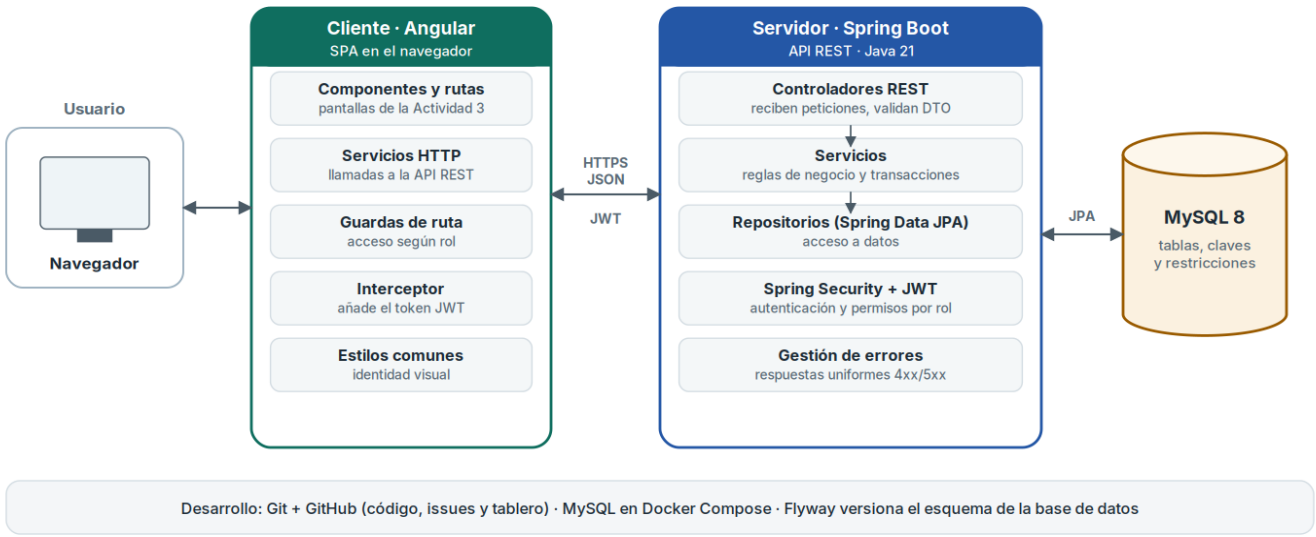
| Actor | Quién es | Casos de uso principales |
|---|---|---|
| **Socio/a** | Persona asociada con cuenta que utiliza material compartido. | Consultar catálogo, ver ficha y disponibilidad, solicitar y cancelar reservas, consultar sus reservas, préstamos e historial. |
| **Responsable de material** | Persona que prepara, entrega, recibe y revisa el material. | Consultar préstamos activos y retrasos, registrar entregas y devoluciones (con incidencia si procede), gestionar inventario, marcar material no disponible, ver el historial de un material. |
| **Administrador/a** | Persona que supervisa el sistema. | Hereda todo lo del responsable y además gestiona usuarios, roles y categorías. |

**Cómo se han usado «include» y «extend»**

**«include»** indica algo que siempre ocurre: solicitar una reserva y marcar un material como no disponible siempre comprueban la disponibilidad. **«extend»** indica algo opcional: una devolución solo registra una incidencia si el material vuelve dañado. Se han usado únicamente donde aclaran la lógica; no es obligatorio que aparezcan.

## 2. Arquitectura general de la solución

Materialia Joven sigue una arquitectura cliente-servidor en tres partes: una aplicación Angular que se ejecuta en el navegador, una API REST desarrollada con Spring Boot y una base de datos MySQL. Cliente y servidor se comunican solo mediante peticiones HTTP con datos en JSON.




*Arquitectura de Materialia Joven*

| Componente | Responsabilidad principal |
|---|---|
| **Cliente Angular** | Mostrar las pantallas de la Actividad 3, recoger los datos del usuario, validar el formato antes de enviarlos y mostrar las respuestas. No contiene reglas de negocio: aunque oculte opciones según el rol, la seguridad real está en el servidor. |
| **Controladores REST** | Recibir cada petición, comprobar que los datos tienen el formato correcto (Bean Validation) y delegar en el servicio correspondiente. |
| **Servicios** | Aplicar las reglas de negocio: disponibilidad, duración máxima, cancelaciones permitidas, cambios de estado. Las operaciones que modifican varias tablas se ejecutan en una transacción. |
| **Repositorios** | Leer y guardar entidades en MySQL mediante Spring Data JPA. |
| **Spring Security + JWT** | Autenticar al usuario y comprobar en cada petición que su rol le permite acceder al recurso (RNF03). |
| **MySQL** | Almacenar los datos de forma persistente y garantizar las restricciones que pueden expresarse en la base de datos (claves, unicidad, comprobaciones de fechas). |

## 2.1 Recursos principales de la API

No se define todavía la API completa, pero sí los recursos principales, porque sirven para repartir el trabajo entre front y back.

| Petición | Qué hace | Rol |
|---|---|---|
| `POST /api/auth/login` | Inicia sesión y devuelve el token | Todos |
| `GET /api/materiales?categoria&disponible` | Catálogo filtrado | Todos |
| `GET /api/materiales/{id}/disponibilidad` | Días ocupados del material | Todos |
| `POST /api/reservas` | Solicita una reserva | Socio/a |
| `GET /api/reservas/mias` | Reservas y préstamos propios | Socio/a |
| `DELETE /api/reservas/{id}` | Cancela una reserva futura | Socio/a |
| `POST /api/reservas/{id}/entrega` | Registra la entrega y crea el préstamo | Responsable |
| `POST /api/prestamos/{id}/devolucion` | Registra la devolución y, si procede, la incidencia | Responsable |
| `GET /api/prestamos/activos` | Préstamos activos, con retrasos marcados | Responsable |
| `POST · PUT /api/materiales` | Alta y edición de material | Responsable |

> **Lo que no hemos añadido**
> No se usan microservicios, colas de mensajes ni varias bases de datos. Para una aplicación de este tamaño añadirían complejidad sin resolver ninguna necesidad de la Fase 1.

# 3. Tecnologías y herramientas

| Ámbito | Tecnología | Por qué tiene sentido en este proyecto |
|---|---|---|
| Frontend | **Angular** + TypeScript | La aplicación tiene muchas pantallas con formularios y tablas, y tres roles con accesos distintos. Angular incluye de serie enrutado, guardas de ruta, formularios con validación y cliente HTTP, por lo que no hay que elegir e integrar librerías por separado. |
| Backend | **Spring Boot** 3 · Java 21 (LTS) | Permite construir la API REST con las capas de la arquitectura y aplicar reglas de negocio con transacciones, que son clave para no aceptar reservas incompatibles. Java es el lenguaje que el equipo domina mejor. |
| Acceso a datos | **Spring Data JPA** (Hibernate) | Las consultas habituales (catálogo, reservas de un usuario) se resuelven sin escribir SQL. Las consultas de solapamiento se escriben de forma explícita para controlarlas mejor. |
| Seguridad | **Spring Security** + JWT | RNF03 exige permisos por rol. Con un token por petición, la API no guarda sesión y el cliente Angular puede enviarlo de forma automática con un interceptor. |
| Base de datos | **MySQL** 8 | Los datos están muy relacionados (usuarios, materiales, reservas, préstamos) y necesitan claves ajenas y restricciones. Es el gestor que el equipo ya conoce. |
| Esquema | **Flyway** | El modelo de datos va a cambiar durante el desarrollo. Con migraciones versionadas, los tres integrantes tienen siempre la misma estructura y los mismos datos de prueba. |
| Entorno | **Docker Compose** (MySQL) | Evita diferencias de instalación entre los equipos de cada integrante. |
| Versiones y tareas | **Git + GitHub** · GitHub Projects | Código, revisiones (pull requests), issues y tablero en el mismo sitio. Cada tarea del tablero se enlaza con su rama y su pull request. |
| Pruebas | **JUnit 5 + Mockito** · Bruno | Pruebas automáticas de los servicios con reglas críticas (disponibilidad) y una colección de peticiones para probar la API manualmente. |

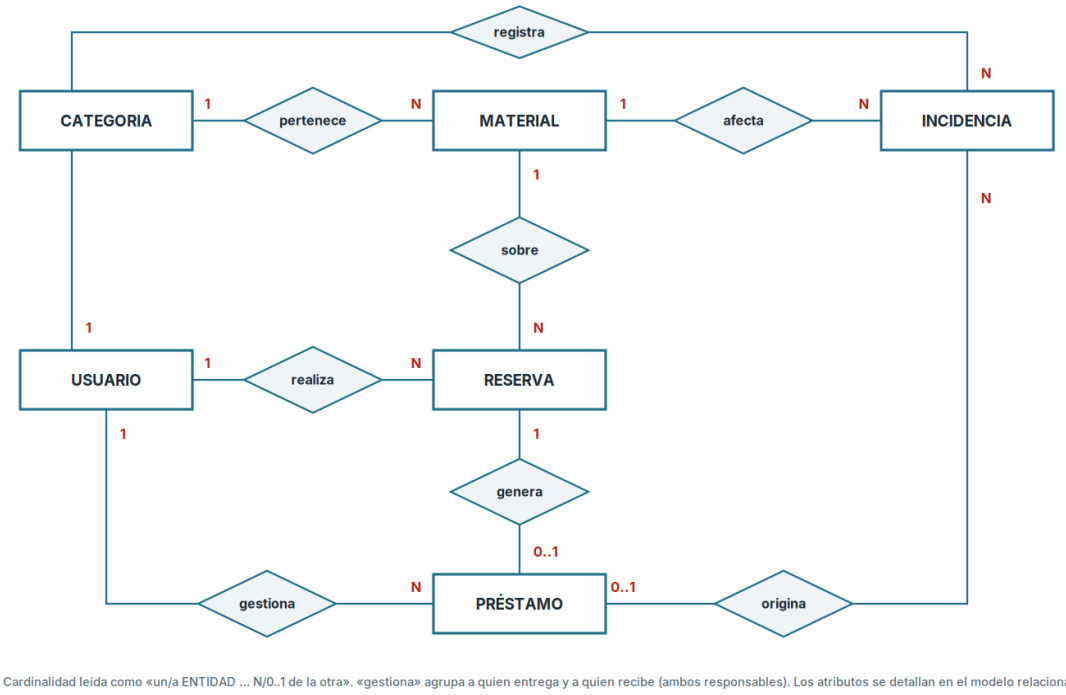
## 3.1 Alternativas que se valoraron

| Alternativa | Por qué no se ha elegido |
|---|---|
| Thymeleaf (vistas generadas en el servidor) | Habría simplificado el inicio, pero el equipo quiere separar cliente y API para poder trabajar en paralelo y probar la API de forma independiente. |
| React o Vue | Válidos, pero exigirían elegir aparte enrutado y gestión de formularios. Con Angular todo viene integrado y el equipo ya lo ha usado. |
| Sesión en servidor en lugar de JWT | Funcionaría, pero con cliente y API separados JWT resulta más sencillo de integrar. |

> **Requisitos técnicos de otros módulos**
> Si el equipo docente comunica requisitos técnicos mínimos que afecten a estas decisiones (por ejemplo, un despliegue concreto o una tecnología obligatoria), se incorporarán y se revisará esta tabla explicando qué cambia y por qué.

# 4. Diseño de datos

## 4.1 Modelo entidad/relación




*Modelo E/R conceptual*

> **Decisión clave: el estado «Disponible / Reservado / Prestado» no se guarda**
> En la Actividad 3 cada material muestra un estado. Guardarlo en una columna obligaría a mantenerlo sincronizado con reservas y préstamos, y cualquier error dejaría datos contradictorios. Por eso **ese estado se calcula** a partir de las reservas y préstamos en vigor. Lo único que se guarda es si el material está operativo o bloqueado por una incidencia (`estado_operativo`). Del mismo modo, «Retrasado» se calcula comparando la fecha prevista con la fecha actual.

## 4.2 Modelo relacional inicial

Clave primaria subrayada · FK = clave ajena · UQ = valor único

| Tabla | Columnas |
|---|---|
| usuario | id, nombre, apellidos, email UQ, password_hash, rol (SOCIO · RESPONSABLE · ADMIN), activo, creado_en |
| categoria | id, nombre UQ, activa |
| material | id, referencia UQ, nombre, descripcion, imagen_url, categoria_id FK → categoria, estado_operativo (OPERATIVO · NO_DISPONIBLE), activo |
| reserva | id, usuario_id FK → usuario, material_id FK → material, fecha_inicio, fecha_fin_prevista, estado (RESERVADA · ENTREGADA · CANCELADA), creada_en |
| prestamo | id, reserva_id FK UQ → reserva, entregado_por FK → usuario, fecha_entrega, recibido_por FK → usuario (nulo hasta la devolución), fecha_devolucion (nula mientras está activo) |
| incidencia | id, material_id FK → material, prestamo_id FK → prestamo (nulo si no surge en una devolución), registrada_por FK → usuario, tipo (AVERIA · REVISION · FALTA_PIEZA · OTRA), observacion, fecha, resuelta |

Cada usuario tiene un único rol, por eso se guarda como columna y no en una tabla aparte. Si más adelante una persona necesitara varios roles, se crearía una tabla intermedia.

## 4.3 Restricciones relevantes

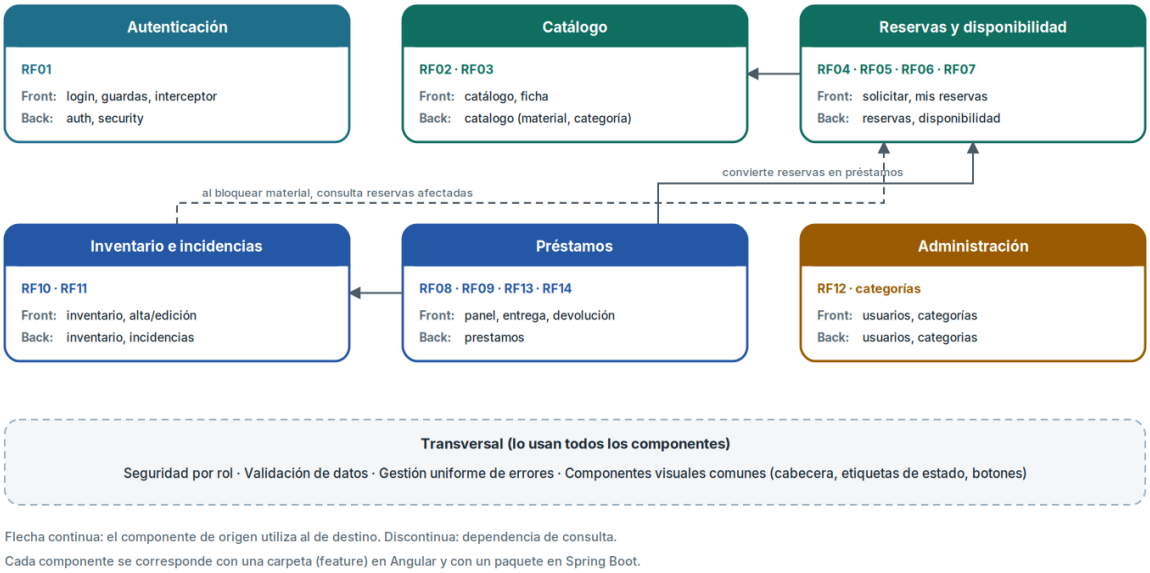
| Restricción | Dónde se garantiza | Relación |
|---|---|---|
| El email de cada usuario es único. | Base de datos (UNIQUE) | RF01 |
| La fecha prevista de devolución no es anterior a la de inicio. | Base de datos (CHECK) y formulario | RF04, RNF04 |
| Un préstamo como máximo de 7 días. | Servicio | RF04 |
| No hay dos reservas activas (RESERVADA o ENTREGADA) del mismo material con fechas que se solapen. | Servicio, dentro de una transacción que bloquea el material mientras se comprueba | RF05 |
| Un material NO_DISPONIBLE no admite reservas nuevas. | Servicio | RF05, RF10 |
| Cada reserva genera como máximo un préstamo. | Base de datos (UNIQUE en reserva_id) | RF08 |
| Solo se cancelan reservas RESERVADA cuya fecha de inicio aún no ha llegado. | Servicio | RF06 |
| Usuarios y materiales no se borran: se desactivan, para conservar el historial. | Diseño (columna activo) | RF11, RF12, RF14 |
| Del usuario solo se guardan nombre, apellidos y email. | Diseño del modelo | RNF05 |

**Por qué el solapamiento no se garantiza solo con una consulta**

Si dos socios reservan el mismo material a la vez, ambas peticiones podrían comprobar la disponibilidad antes de que la otra guarde, y las dos reservas se aceptarían. Por eso la comprobación y el guardado se hacen en la misma transacción, bloqueando la fila del material hasta terminar. Esta regla tendrá pruebas automáticas propias.

# 5. Componentes o áreas funcionales

La aplicación se divide en seis componentes con una responsabilidad reconocible. Cada uno agrupa requisitos, pantallas y código, y servirá de base para repartir las tareas de desarrollo.




*Mapa de componentes funcionales*

La misma división se refleja en la organización del código:

```
materialia-api/src/main/java/.../materialia/
├── auth/          ├── catalogo/      ├── reservas/
├── prestamos/     ├── inventario/    ├── usuarios/
└── comun/         (seguridad, errores, configuración)

materialia-web/src/app/
├── core/          (interceptor, guardas, servicio de sesión)
├── shared/        (cabecera, etiqueta de estado, botones)
└── features/      catalogo/ · reservas/ · prestamos/ · inventario/ · admin/
```

| Componente | Responsabilidad | Relación con otros componentes |
|---|---|---|
| **Autenticación** | Inicio de sesión, emisión del token, control de acceso por rol. | Todos dependen de él. |
| **Catálogo** | Consultar materiales y categorías y calcular el estado que se muestra. | Proporciona los materiales que se reservan. |
| **Reservas y disponibilidad** | Crear, consultar y cancelar reservas. Contiene el servicio de disponibilidad. | Usa el catálogo; préstamos e inventario lo consultan. |
| **Préstamos** | Entregas, devoluciones, préstamos activos, retrasos e historial. | Convierte reservas en préstamos y crea incidencias. |
| **Inventario e incidencias** | Alta y edición de material, bloqueo y registro de incidencias. | Al bloquear, consulta las reservas afectadas. |
| **Administración** | Usuarios, roles y categorías. | Independiente del flujo de préstamo. |

# 6. Secuencia de procesos clave

Se han elegido los dos procesos con más lógica: la reserva, porque concentra la regla más importante del sistema (no aceptar reservas incompatibles), y la devolución con incidencia, porque modifica varias tablas en una sola operación.

## 6.1 Solicitar una reserva

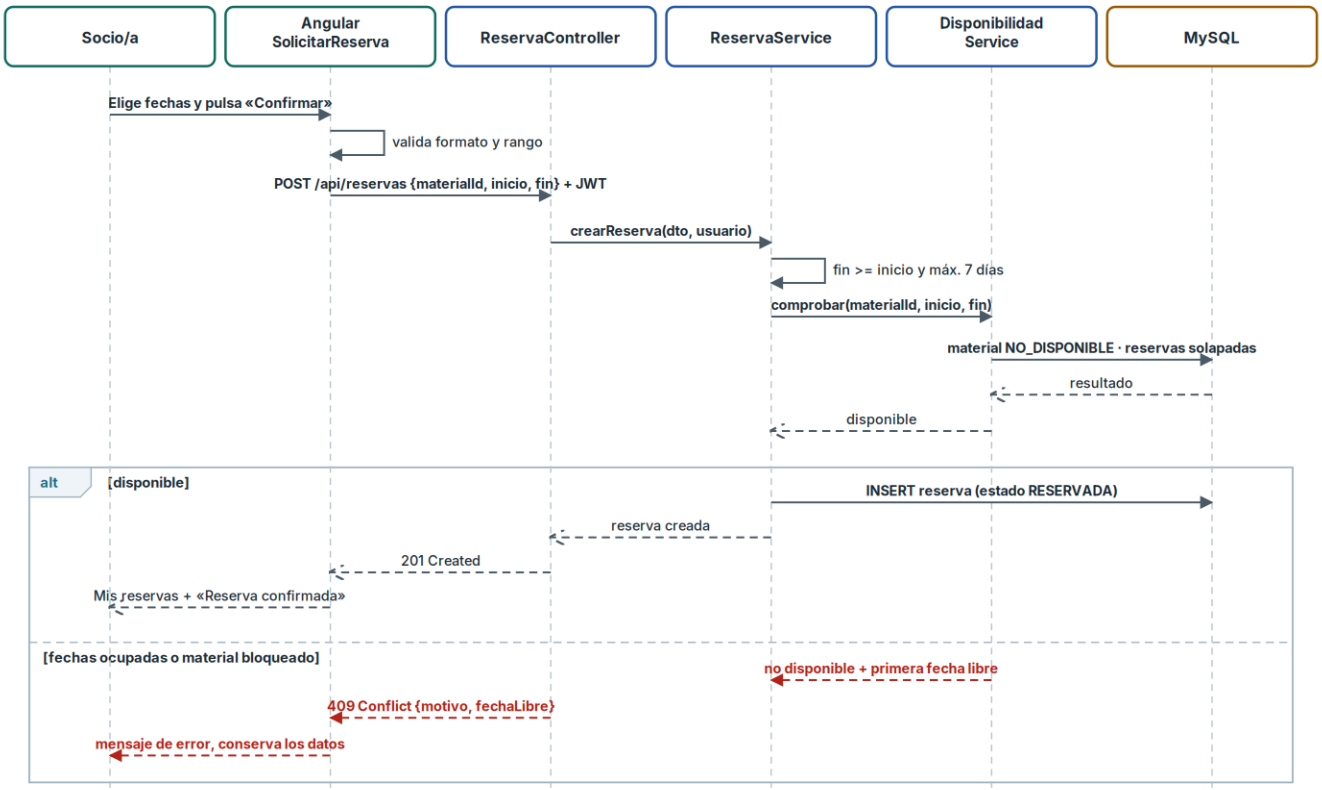



*Diagrama de secuencia: solicitar reserva*

El cliente valida el formato de las fechas para dar una respuesta rápida, pero la decisión la toma siempre el servidor. El servicio comprueba las reglas de la reserva y delega en el servicio de disponibilidad, que reutilizan también otros componentes. Si hay conflicto, la API responde con `409 Conflict` e incluye el motivo y la primera fecha libre, que es lo que muestra el mensaje diseñado en el wireframe 4.3 de la Actividad 3.

## 6.2 Registrar una devolución con incidencia

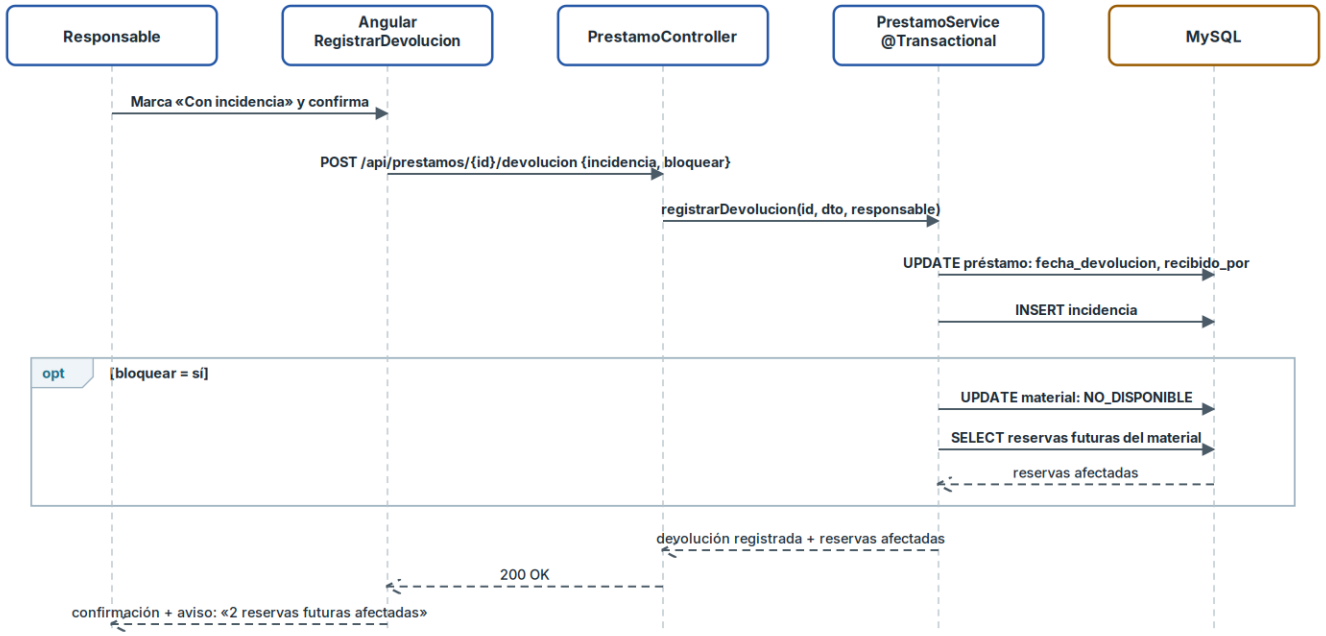



*Diagrama de secuencia: devolución con incidencia y bloqueo*

Cerrar el préstamo, registrar la incidencia y bloquear el material se hacen en una única transacción: si falla una parte, no se guarda ninguna y los datos no quedan a medias.

> **Cuestión pendiente de la Actividad 3, resuelta**
>
> Si el material bloqueado tenía reservas futuras, el sistema **no las cancela automáticamente**: devuelve la lista al responsable para que decida (reparar a tiempo, ofrecer otro material o cancelar). Es la opción más sencilla y evita cancelar reservas por una incidencia que quizá se resuelve en un día. La cancelación automática con aviso al socio/a queda como posible mejora.

# 7. Definición del MVP

El MVP de Materialia Joven es la versión más pequeña que permite completar el **flujo principal definido en la Actividad 1: consulta, solicitud, entrega y devolución**, incluyendo el bloqueo de material dañado. Con él se puede comprobar si la aplicación resuelve el problema de partida.

| Funcionalidad | RF | Componente | Fase | Justificación |
|---|---|---|---|---|
| Iniciar y cerrar sesión con roles | RF01 | Autenticación | MVP | Sin ella no se distinguen socios y responsables. |
| Catálogo con filtros por categoría y disponibilidad | RF02 | Catálogo | MVP | Es la pregunta que antes se hacía por mensaje. |
| Ficha con estado y calendario | RF03 | Catálogo | MVP | Necesaria para elegir fechas válidas. |
| Solicitar reserva sin solapamientos | RF04, RF05 | Reservas | MVP | Núcleo del proyecto. |
| Consultar mis reservas y préstamos | RF07 | Reservas | MVP | El socio/a debe ver el resultado de su solicitud. |
| Registrar entrega | RF08 | Préstamos | MVP | Convierte la reserva en préstamo. |
| Registrar devolución con incidencia y bloqueo | RF09, RF10 | Préstamos, Inventario | MVP | Cierra el flujo y evita reservar material averiado. |
| Panel de préstamos activos y retrasos | RF13 | Préstamos | MVP | Prioridad media, pero es la pantalla de entrada del responsable y su coste es bajo. |
| Alta y edición de material | RF11 (parte) | Inventario | MVP | Sin ella no hay forma de cargar material real. |
| Cancelar reserva futura | RF06 | Reservas | Después | Útil, pero el flujo principal funciona sin ella. |
| Desactivar material | RF11 (parte) | Inventario | Después | En el MVP basta con marcarlo no disponible. |
| Gestión de usuarios y roles | RF12 | Administración | Después | En el MVP las cuentas se cargan mediante una migración de datos iniciales. |
| Gestión de categorías | — | Administración | Después | Las categorías son pocas y estables; se cargan igual que las cuentas. |
| Historial completo por material y usuario | RF14 | Préstamos | Después | Los datos ya se guardan desde el MVP; solo falta la consulta. |
| Buscador por texto en el catálogo | RF02 (ampliación) | Catálogo | Después | Los filtros son suficientes con el material de prueba. |

## 7.1 Qué puede hacer cada usuario con el MVP

| | |
|---|---|
| **Socio/a** | Entra con su cuenta, encuentra material disponible, consulta en qué fechas está libre, lo reserva y ve su reserva y sus préstamos en curso. |
| **Responsable** | Ve qué tiene que entregar hoy y qué préstamos están activos o retrasados, registra entregas y devoluciones, anota incidencias, bloquea material dañado y da de alta material nuevo. |
| **Administrador/a** | Puede hacer lo mismo que el responsable. La gestión de cuentas y categorías llega después del MVP. |

> **Comprobación del MVP**
> El MVP se considerará alcanzado cuando los recorridos R1, R2 y R3 de la Actividad 3 se puedan completar de principio a fin con datos de prueba, incluido el caso de reserva con fechas ocupadas.

# 8. Planificación inicial del desarrollo

## 8.1 Tareas hasta el MVP

| ID | Tarea | Componente | Depende de | Responsable | Semanas |
|---|---|---|---|---|---|
| T01 | Repositorio, tablero y normas de trabajo | — | — | Equipo | 1 |
| T02 | Proyecto Spring Boot + MySQL (Docker) + Flyway | Común | T01 | Hugo | 1 |
| T03 | Proyecto Angular: rutas, layout e identidad visual | Común | T01 | Sara | 1–2 |
| T04 | Esquema inicial y datos de prueba | Común | T02 | Nerea | 2 |
| T05 | Autenticación JWT y roles (back) | Autenticación | T02, T04 | Hugo | 2–3 |
| T06 | Login y guardas de ruta (front) | Autenticación | T03, T05 | Sara | 3 |
| T07 | API de catálogo y categorías | Catálogo | T04 | Nerea | 3 |
| T08 | Catálogo y ficha (front) | Catálogo | T06, T07 | Sara | 4 |
| T09 | Inventario: alta, edición y no disponible | Inventario | T07 | Nerea | 4 |
| T10 | Reservas y disponibilidad (back + pruebas) | Reservas | T05, T07 | Hugo | 4–5 |
| T11 | Solicitar reserva y Mis reservas (front) | Reservas | T08, T10 | Sara | 5 |
| T12 | Panel, entrega y devolución con incidencia | Préstamos | T09, T10 | Nerea / Hugo | 5–6 |
| T13 | Integración y pruebas de R1, R2 y R3 | Todos | T11, T12 | Equipo | 6–7 |
| T14 | Revisión del MVP y presentación | — | T13 | Equipo | 8 |

Las tareas del tablero son más pequeñas que estas: por ejemplo, T10 se divide en «consulta de solapamiento», «servicio de reservas», «controlador» y «pruebas de disponibilidad».

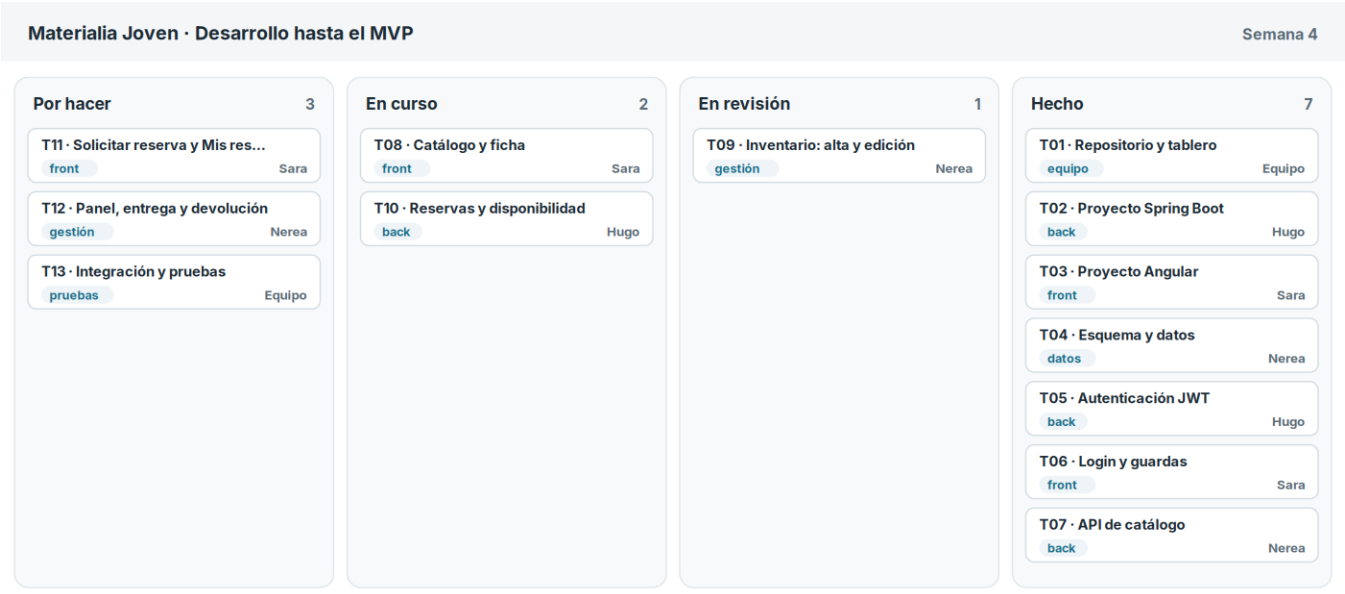
## 8.2 Diagrama de Gantt simplificado




*Previsión inicial hasta el MVP. Se revisará al final de cada semana.*

El camino crítico es **T02 → T05 → T10 → T12 → T13**: un retraso en la autenticación o en las reservas retrasa el MVP. Por eso T10 la asume quien ya ha montado el backend y empieza en cuanto la API de catálogo está lista.

## 8.3 Organización en el tablero




*Tablero del equipo en la semana 4 (imagen ilustrativa: en la entrega real se incluye una captura del tablero del equipo)*

| Norma | Cómo se aplica |
|---|---|
| Una tarea, una issue | Cada tarjeta es una issue con responsable, componente y criterio de terminado. |
| Una issue, una rama | Se trabaja en una rama por tarea y se integra mediante pull request. |
| Revisión cruzada | Ningún pull request se integra sin que lo revise otra persona del equipo. |
| «Hecho» significa… | Integrado en la rama principal, probado y, si es back, con su petición en la colección de Bruno. |
| Revisión semanal | Cada semana se actualizan el tablero y, si hace falta, el Gantt. |

## 8.4 Riesgos previstos

| Riesgo | Medida |
|---|---|
| La autenticación JWT lleva más tiempo del previsto. | Empezar en la semana 2 y, si se bloquea, desarrollar el resto con usuarios de prueba fijos. |
| Front y back no encajan al integrar. | Acordar los recursos de la API (apartado 2.1) antes de programar y probar cada petición con Bruno. |
| Errores en la regla de solapamiento. | Pruebas automáticas desde T10 con casos límite (mismo día, fechas contiguas, material bloqueado). |

## 9. Qué se presentaría al finalizar la Actividad 4

| Bloque | Qué debería explicarse |
|---|---|
| Casos de uso | Actores y casos principales, y cómo encajan con los usuarios de la Fase 1. |
| Arquitectura | Partes de la solución y responsabilidad de cada una. |
| Datos | Entidades, relaciones y restricciones; por qué el estado del material se calcula. |
| Tecnologías | Por qué tienen sentido para este proyecto, no cuáles son «las mejores». |
| Componentes | Cómo se divide la aplicación y cómo se relaciona con el código y el reparto. |
| Secuencia | Un proceso explicado paso a paso: la reserva. |
| MVP | Qué entra, qué queda fuera y cómo se comprobará que se ha alcanzado. |
| Planificación | Gantt, camino crítico, reparto y tablero. |

**Síntesis posible para el cierre oral**

Materialia Joven será una aplicación Angular que consume una API REST en Spring Boot con MySQL. El modelo tiene seis entidades y el estado visible del material se calcula para que nunca contradiga a las reservas. La regla más delicada, impedir reservas solapadas, se resuelve en el servidor dentro de una transacción y tendrá pruebas propias. El MVP permite completar el ciclo consulta, reserva, entrega y devolución con incidencias, y queda fuera la gestión de usuarios, categorías e historial. Hemos planificado ocho semanas con catorce tareas; el camino crítico pasa por autenticación y reservas, y el trabajo se sigue en GitHub Projects.

## 10. Comprobación final del ejemplo

- Los actores del diagrama de casos de uso coinciden con los usuarios de la Actividad 2.
- La arquitectura solo incluye las partes necesarias y explica la responsabilidad de cada una.
- Cada tecnología está justificada por una necesidad del proyecto.
- El modelo de datos cubre todos los requisitos y sus restricciones indican dónde se garantizan.
- Los componentes se corresponden con requisitos, pantallas y organización del código.
- Los diagramas de secuencia explican procesos con lógica real, incluidos los casos de error.
- El MVP completa los recorridos esenciales y lo que queda fuera está justificado.
- La planificación tiene dependencias, responsables, Gantt y tablero coherentes entre sí.

**Resultado esperado**

Al terminar la Fase 2 el equipo sabe cómo será la aplicación, cómo se construirá y qué debe desarrollar primero. Muchas de estas decisiones cambiarán durante el desarrollo; cuando lo hagan, habrá que poder explicar qué se ha modificado y por qué.

**HITO F2: el proyecto queda preparado para comenzar su desarrollo.**

Este documento es un ejemplo de profundidad y forma de justificar decisiones. El proyecto del alumnado deberá ser propio y responder a su necesidad concreta.